

This small calculation enables us to use a 1D array or list to store the values of a 2D tile board. This programming technique is useful not just in our project but for any 2D data structure that must be stored in an array or list, such as a 2D image stored as a stream of bytes.

0,0	1,0	2,0	3,0
0,1	1,1	2,1	3,1
0,2	1,2	2,2	3,2
0,3	1,3	2,3	3,3

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Figure 12-5: The x, y coordinates for each space on the board (left) and the corresponding data structure index (right)

Let's look at some example data structures. The board with mixed-up tiles shown previously on the left side of [Figure 12-1](#) would be represented by the following:

```
[15, 2, 1, 12, 8, 5, 6, 11, 4, 9, 10, 7, 3, 14, 13, 0]
```

The solved, ordered puzzle on the right side of [Figure 12-1](#) would be represented by this:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0]
```

All the functions in our program will expect board data structures that follow this format.

Unfortunately, the 4×4 version of the sliding-tile puzzle has so many possible moves that it would take a normal laptop weeks to solve. You can change the `SIZE` constant from 4 to 3 to solve a simpler 3×3 version of the puzzle. The finished, ordered 3×3 puzzle's data structure would look like this:

```
[1, 2, 3, 4, 5, 6, 7, 8, 0].
```